

helix-code-1.2.0-dev-0.0.1

HelixCode Release Changelog — helix-code-1.2.0-dev-0.0.1

Revision: 5 **Last modified:** 2026-08-10T21:43:44Z **Maintainer:** HelixCode release engineering (documentation draft, AI-agent authored; revisions 1–3 independently reviewed per §11.4.142/§11.4.194 — see “Independent review” section below. Revisions 4–5 are verified-by-construction against `git log/git show/docs/workable_items.db` but have **not** themselves crossed a §11.4.142 independent-reviewer seam — see the “Independent review” section’s revision 4/5 entries for the honest boundary.)

DRAFT — pinned to a stated HEAD, this is not a cut tag. No git tag has been created by this document. Everything below was derived by reading real git history (`git log`, `git show`) — revision 1–2 content on 2026-07-28, the revision 3 content on 2026-08-09 at 12:38 UTC. Multiple background work streams were actively landing commits in this repo and its submodules while this draft was written (constitution §11.4.103 / §11.4.192 continuous multi-track work), so HEAD will have moved by the time this is read. **Re-derive the commit ranges below before treating any of this as final** — do not copy the SHAs or counts forward without checking.

Repo	Stated HEAD (revision 4)	Relative to its configured remotes
main (this repo)	84853455bf20c940cc32687918c6a7cc42cfda81	10 ahead, 0 behind on all four remotes (github/ gitlab/ origin/ upstream) — measured <i>after</i> a real

Repo	Stated HEAD (revision 4)	Relative to its configured remotes
		git fetch --all --prune on 2026-08-10 (§11.4.6)
constitution (pin)	177f2b05545f2fe73fbb432ad0de3a6bed7edcf4	unchanged since revision 3; clean
submodules/helix_llm (pin)	3d789cb888a09cfecdd2f0007a08a96d91caf5e9	unchanged since revision 3; clean
submodules/helix_agent (pin)	94e2dcc828ef31930aa2926584ffae273e58a04c	resolves revision 3's Known gap 14 — the pin now equals the submodule's own current HEAD exactly (git submodule status shows no +); 24 ahead, 0 behind on all four of its own remotes, measured after a real fetch
submodules/tool_schema (pin)	8cec90baadad281eb26a5671b792921f03eb7386	unchanged since revision 3; clean

Revision 4 scope. Re-derives from 5725ee06 . . 84853455 in the main repo — 7 **commits**, of which 2816246e is the commit that *wrote* revision 3 of this document (already fully self-describing; not re-described here) and the other 6 (04f88a23, be5d56be, 69573772, b947276f, f10c9a1e, 84853455) are new substantive content. It also fully characterises submodules/helix_agent’s entire 66a3c1c6 . . HEAD range — **24 commits** — which revision 3 recorded only as a pin/checkout mismatch (Known gap 14) and explicitly declined to enter or describe; none of those 24 commits appear anywhere in revisions 1–3 of this document. Total since the last tag helix-code-1.1.0-dev-0.0.3 (5ab97d8c) in the main repo: **256 commits** (git rev-list --count 5ab97d8c . . HEAD — 249 from revisions 1–3, plus this section’s 7); **0 merge commits** anywhere in that range; **0** Auto-commit-only messages in the revision 4 range specifically (both repos), a positive change from the pattern revisions 1–3 flagged (Known gaps 8, 16). Revisions 1–3 content below is **preserved, not rewritten**.

Revision 3 scope. Revisions 1–2 documented 5ab97d8c . . c29e1dcc. Revision 3 re-derives from c29e1dcc . . 5725ee06 — a further **231 commits** spanning 2026-07-28 → 2026-08-09, of which exactly **2** are the literal Auto-commit message and 229 are substantive; **0 merges** anywhere in the release range. Total since the last tag helix-code-1.1.0-dev-0.0.3 (5ab97d8c): **249 commits** (18 + 231, and git rev-list --count 5ab97d8c . . HEAD independently returns 249). The revision 1–2 content below is **preserved, not rewritten** — it was independently reviewed and its claims re-verified; revision 3 adds the new range, corrects one counting error revision 2 introduced (see “Independent review”), and refreshes every pin and remote position.

Format note: docs/changelogs/ did not exist in this repository before this draft — it was created now. The repository’s existing root CHANGELOG.md uses a ## [tag] – date per-release section with Added / Changed / Fixed / Known-gaps groups; this draft follows that convention but adds explicit short-SHA citations per entry and a per-repo commit inventory, so every claim traces back to a real, readable commit (constitution §11.4.6 — no invented history). Per the task scope, nothing here was committed with git add -A or similar — only this one file is staged, by explicit path.

Version

helix-code-1.2.0-dev-0.0.1 — draft, not yet tagged.

Prefix resolves to helix-code from .env HELIX_RELEASE_PREFIX (§11.4.151).

Prior tags in this lineage:

Tag	Date
helix-code-1.0.0-dev-0.0.1	2026-07-08
helix-code-1.1.0-dev-0.0.1	2026-07-09
helix-code-1.1.0-dev-0.0.2	2026-07-13
helix-code-1.1.0-dev-0.0.3	2026-07-24

On the version jump: helix-code-1.2.0-dev-0.0.1 is a **1.1.x → 1.2.0 minor bump**, not the next -1.1.0-dev-0.0.4 patch in the existing sequence. This was named explicitly by the operator as this release's target tag; per instruction for this task, that is recorded here as an **operator decision**, not re-litigated or treated as unexplained drift.

Version-number determination (revision 4, re-checked from commit content per task instruction). Independent of the operator's naming decision above, the commit content of the full 5ab97d8c . . HEAD range (256 commits, all three revisions combined) supports a minor bump on its own evidentiary merits, not merely a patch increment:

- **New user-visible capability, not just fixes.** Revision 3's range alone contains 9 feat commits landing genuinely new capability: the entire systemd-unit platform installer (c4115c73) standing up 5+ services from a single ./setup.sh; LLMsVerifier wired in as a 7th unit with real provider scoring (d3e2c6e7, 98db7d79); and Hyper provider discovery in submodules/helix_agent (410afda5). A restored primary capability also belongs in this bucket, not the patch bucket: HXC-233 (revision 3) fixed the gateway's *only* completion path when no cloud key is configured — every completion request was returning HTTP 500 before that fix, which is a product-defining regression-and-restoration, not a minor patch-grade fix.
- **Scale.** 256 commits across two repositories, spanning roughly two weeks (2026-07-28 → 2026-08-11), is far outside the size of a typical -dev-0.0.N patch increment in this project's own tag history (the three prior 1.1.0-dev-0.0.N patches were each single-digit-to-low-double-digit commit counts by comparison).
- **Governance/architecture-level changes**, which a semver-conscious reading would also weigh toward minor-or-above: the root module rename (dev.helix.code → dev.helix.code/meta, HXC-187/D-7, 09a086a6) and the workable-items tracker's SQLite-backed re-architecture reaching its own standing-guard-wiring maturity milestone in this range (be5d56be, 32 gates up from 29).
- **Against a patch reading:** the revision-4-specific range documented in this update (the 6 new main-repo commits plus the 24 submodule commits) is, on

its own, mostly `fix/test/docs/items` work with no new feat commit — if that range were being version-numbered in isolation, a patch bump would be the better fit. It is not being numbered in isolation: it is additional content folding into the same `1.2.0-dev-0.0.1` release the operator already named, alongside the feat-bearing revision 1–3 range.

Conclusion: the evidence supports keeping `1.2.0-dev-0.0.1` as named. This document does **not** change the version number — per the task’s explicit instruction, and independent of that instruction, the target file names (`docs/changelogs/helix-code-1.2.0-dev-0.0.1.{md,html,pdf}`) already reference it, so unilaterally renaming the release here would create exactly the kind of un-synced reference (§11.4.186) this document exists to prevent. If a future reviewer disagrees with the minor-vs-patch call, the correct fix is to rename the draft’s own files and every reference to them in one atomic change, not to edit the version string inside a file whose name still says otherwise.

Summary — what this release actually delivers for a user

- **Model identity is now honest end-to-end.** A client asking for an alias (e.g. `"model": "default"`) previously got that same alias echoed back in the response, on both the OpenAI-compatible and Anthropic-compatible wires, streaming and non-streaming. The response now names the model that actually served the request (e.g. the real `.gguf` path), matching `CONST-036/037`. Live SSE evidence was captured against a real running coder backend for both wires. One honest exception remains: Anthropic’s streaming `message_start` event fires before any chunk exists, so it still carries the requested model — documented in-source, not silently left as a gap.
- **tool_schema translations now work regardless of where you run from.** Previously, calling into this library’s `i18n` from any working directory other than its own module root silently returned raw translation keys instead of translated text (e.g. `toolschema_git_desc_commit` instead of “Create git commit”). Bundles are now compiled into the binary (`go:embed`), so this can no longer happen.
- **A large batch of false test failures in helix_agent is fixed.** A liveness check used to accept *any* HTTP response under 500 as “the server is up” — so when a different local service (LLMsVerifier) held the same default port, its 404 response was read as a healthy HelixAgent, and roughly a dozen test packages failed against the wrong service with nothing actually wrong in the product.
- **The whole platform can now be installed and booted as systemd units** (`./setup.sh`) — HelixCode server, HelixAgent, HelixLLM gateway + coder, infra (Postgres/Redis/Weaviate/Qdrant/ChromaDB/Cognee/Ollama), and now

LLMsVerifier — with real HTTP health checks and a verified stop/start cycle, rather than requiring manual, host-specific setup.

- **LLMsVerifier is wired to real provider data and publishing real scores** instead of an empty `{}` — the gateway now routes on measured scores instead of always falling back to its static table.
- **`go test ./...` no longer breaks on generated challenge output** — a gitignored-but-still-scanned results directory could fail three otherwise unrelated packages depending on what an LLM happened to answer during a challenge run.
- **helix_agent boot no longer requires ChromaDB/Cognee on their compiled-in ports**, and no longer hangs indefinitely trying to start a vector store that is already running under a different owner (committed in helix_agent but see the pinning note below).

Fixed

SHA	Repo	Summary
e330149f	main	LLM response now reports the model that actually served the request (OpenAI + Anthropic wires, streaming + non-streaming), not the requested alias. Four layers: <code>llm.LLMResponse.Model</code> field + JSON round-trip, <code>OpenAICompatibleProvider</code> populating it from both non-stream and per-chunk stream responses, and both wire facades preferring the served model. Guards at <code>internal/server/wire_facade_model_identity_test.go</code> and <code>internal/llm/openai_compatible_stream_model_test.go</code> , each proven to fail when its fix is reverted.
66d6fb29	main	<code>go test ./...</code> no longer descends into <code>tests/e2e/challenges/test-results/</code> (renamed to <code>_test-results/</code> , the Go toolchain's own “not a package” convention) and tries to compile an LLM's prose answer as Go source. Unblocked 3 packages.
3268fd73	helix_agent (pinned)	<code>testutil.ServerAvailable</code> now requires a real HelixAgent-shaped health payload (HTTP 200 + identity check), not just “any status under 500”. Fixes a false-failure cluster of roughly a dozen packages caused by a different local service (LLMsVerifier) answering on the same

SHA	Repo	Summary
		default port. Three additional call sites that bypassed the shared helper entirely (a raw TCP dial, an error-only skip check) were also fixed.
0165ab1d	helix_agent (<i>pinned</i>)	Precondition gate's container-ownership check fixed in both directions: it no longer flags an unrelated third-party Postgres as "our" duplicate instance (false positive from {port, port+10000} guessing), and it no longer silently no-ops because it ran the wrong runtime command (false negative). Ownership is now established from the actual container runtime, filtered to this project's own containers.
0dfa69e1	helix_agent (<i>pinned</i>)	HelixAgent boot no longer hardcodes ChromaDB (:8001) / Cognee (:8000) endpoints — both resolve via HELIXAGENT_DEP_HOST / HELIXAGENT_PORT_CHROMADB / HELIXAGENT_PORT_COGNEE with the old literals as fallback. Vector-store startup now checks reachability first instead of unconditionally trying to become a second owner of an already-running store, which had been blocking the HTTP listener bind indefinitely.
8cec90b	tool_schema (<i>pinned</i>)	Translation bundles are go:embed-ed, so tr() lookups no longer depend on the process's current working directory. Previously any consumer calling in from outside this module's own root got raw key strings back instead of translations.
69a5ab8d	helix_agent (committed, NOT yet pinned — see Known gaps)	Fixes a goroutine leak in the Ollama provider's CompleteStream: 4 of 6 channel-send sites had no ctx.Done() escape hatch, so a caller that cancelled its context and stopped draining left the background goroutine parked forever. RED/GREEN regression guard added, driven via a real stuck-HTTP-response scenario and a live goroutine-stack inspection.
a345c551	helix_agent (committed, NOT yet	Two tests/unit test defects fixed (product behavior was correct in both cases): TestCompletionHandler_Models now wires a real model source instead of asserting on an

SHA	Repo	Summary
	pinned — see Known gaps)	intentionally-empty list; TestCompletionHandler_Stream now distinguishes an honest empty response (client-gone, request-timeout path) from an actual failure, instead of treating the httptest recorder’s default 200 as an unconditional pass signal.
dd3c0c3b	main	Closes the third confirmed instance of an “unprioritized select” race this session: autoSaveLoop’s per-tick select between <- ticker.C and <- stop had no priority ordering, so a coincidentally-pending tick could still fire one more SaveAll() after DisableAutoSave() had already closed stop — the cause of TestAutoSaveLifecycle_ReEnableTicks’s “disabled auto-save must not tick” flake. Fixed by extracting the per-iteration select into autoSaveTick(tickerC, stop) with a non-blocking priority pre-check on stop. Full RED/GREEN/pre-fix/post-fix polarity evidence captured (§11.4.115): 2488/5000 forced-race hits pre-fix with the priority check disabled, 0/5000 post-fix. Found missing from this changelog’s original draft during independent review — added in revision 2; see “Independent review” section below.

Added

SHA	Repo	Summary
c4115c73	main	. /setup.sh now installs and boots the entire platform as systemd units (helix.target + 5 services) — HelixCode server, HelixAgent, HelixLLM gateway, HelixLLM coder, and infra — with a

SHA	Repo	Summary
		documented port map, boot-persistence via Linger=yes, and a verified full stop/start cycle. Fixes a KillMode=control-group bug that had been silently killing all ten infra containers' port forwarders whenever the infra unit's own health check failed.
d3e2c6e7	main	Adds LLMsVerifier as a 7th systemd unit, wired to the gateway. Implements the /api/scores endpoint upstream in the submodule (project-agnostic; consumer supplies the port via the unit). Gateway's log changed from "verifier unreachable" to actually reaching the verifier and correctly falling back on an empty score map.
98db7d79	main	LLMsVerifier is now seeded with a real, reachable provider (llamacpp → the local llama.cpp coder server) via a consumer-owned config/llmsverifier/config.yaml, instead of the submodule's own empty llms: [] default. First real verification score published (0.96) from genuine inference,

SHA	Repo	Summary
		replacing the previously-empty <code>/api/scores</code> .
410afda5	helix_agent (<i>pinned</i>)	Hyper provider discovery, registry, and verifier types.
1296d328	main	Adds scripts/ <code>setup.sh</code> scaffolding (157 lines) and two systemd unit files (<code>helixcode-infra.service</code> , <code>helixllm-gateway.service</code>) — precursor work later built out by c4115c73 / d3e2c6e7. Commit message is the literal <code>Auto-commit</code> ; see Known gaps.

Changed

SHA	Repo	Summary
925aa859	main	Backfills the constitution's §11.4.182 TRUNK RULE (work on trunk is always labelled Track 1) across this project's five root governance carriers, and bumps the <code>helix_agent</code> and <code>tool_schema</code> submodule pins to the commits described above.
f14a3f89	main	Bumps the <code>helix_agent</code> submodule pointer (Hyper provider).
565506f6	main	

SHA	Repo	Summary
		Bumps HelixLLM to helixllm-0.2.0 (CPU inference).
c20215bd	helix_agent (<i>pinned</i>)	Chaos/compliance/automation test hardening carried in from prior sessions and committed so it is not lost — chaos/api, chaos/auth, module-compliance, config-generation automation, plus new helix_endpoint contract tests. go build ./... verified exit 0; the commit itself is explicit that the full suite was not independently reviewed or proven green at the time.
4b5f2867	helix_agent (<i>pinned</i>)	gofmt alignment and comment-spacing pass across the module.
36597718	helix_agent (<i>pinned</i>)	Search test reconciled to a reordered UnknownType check; adds coverage for the reachability short-circuit.

Infrastructure / Governance / QA evidence

SHA	Repo	Summary
578de68d	main	Lands §11.4.83 end-user QA evidence for 7 feature commits, captured against a freshly rebuilt binary (111 assertions, 0 FAIL, 0 SKIP).

SHA	Repo	Summary
		Explicitly documents that an earlier evidence pass had validated a 15-day-stale binary and was therefore invalid and re-run.
1c9fdcf2	main	§11.4.83 evidence against live paid providers (Cerebras, Cohere) and a real MCP wire for 3 further commits; drops the qa-evidence-gate violation count from 6 to 3.
e952f4d1	main	§11.4.83 evidence for the last 3 non-feature commits (CORS allowlist, Fyne desktop build docs, cognee determinism), each carrying an explicit “honest scope statement” about limited/no end-user surface. Drives the qa-evidence gate (G7) from 13 to 0 violations across 28 feature-shipping commits.
d8c44618	main	§11.4.83 evidence specifically for the e330149f model-identity fix (8/8 assertions PASS, mutation table proving each guard fails when deleted).
a5462e35	main	Live SSE proof, against the real coder backend, that <code>stream:true</code> reports the served model on every chunk (13 frames captured, 1 distinct model id, 0 frames carrying the alias).
c29e1dcc	main	Live proof for the Anthropic wire (POST /v1/messages returns the real served-model path) and for the tool_schema i18n fix (a compiled consumer binary resolves translations

SHA	Repo	Summary
		from unrelated working directories).
cf26a173	main	Bumps 4 owned submodule gitlinks (constitution, challenges, helix_qa, helix_agent) after pushing them to their upstreams. Commit itself states the full test sweep was not green at the time and no tag was cut.
0a4eb8d0	main	Corrects stale live-state anchors in RESUME.md after the above push — the exact “stale handoff doc” trap the project’s own governance warns about, caught and fixed in the same session.
56f7edf7	main	Message is the literal Auto-commit (no descriptive body). Diffstat shows real, substantive content: nil-request guards for the Replicate and Together LLM providers, a malformed-URL guard for the llamacpp provider, a security_scan tool rewrite, and an overhaul of verify_qa_evidence.sh / verify-governance-cascade.sh / obsolete_details_gate.sh / summary_sync_gate.sh, plus a helix_agent pointer bump. See Known gaps.

Revision 3 — the c29e1dcc..5725ee06 range (231 commits, 2026-07-28 → 2026-08-09)

Everything from here to “Per-repo commit inventory” was derived on 2026-08-09. Commit-type histogram over the 229 substantive commits: 87 docs, 74 fix, 27

chore, 15 test, 9 feat, 7 items, 3 track, 2 governance, 1 probe, 1 docs+test.
72 distinct HXC ids are named in commit subjects (HXC-151...HXC-248).

Summary — what changed for a user in this range

- **The gateway’s primary capability was dead and now works.** Every completion returned HTTP 500 “all providers exhausted”: with no cloud keys configured the local provider was the only completion path, and it was pointed at port 50052 — a default matching no service in the deployment, while the local model is served on 18434. Completions now return 200 with real model output.
- **The health endpoint no longer reports a verdict it never earned.** It returned `{"status": "healthy", "components": []}` and could return nothing else. It now names four checked dependencies, each with a measured round-trip duration.
- **You can now tell semantic search from a hash fallback.** The embeddings response previously carried no signal at all about which pipeline served it.
- **The desktop application no longer corrupts its own UI state under load** — 265 data races eliminated, root-caused to a *false* code comment claiming a widget setter was goroutine-safe when the toolkit’s own source says the opposite.
- **Streaming-with-tools no longer deadlocks** — a guaranteed (not racy) hang, plus a family of provider send-leaks that stranded a goroutine and an open HTTP body whenever a client disconnected.
- **Infrastructure “started” now means “serving”, not “created”** — the boot path previously reported success once containers existed, before anything answered.
- **Two credential exposures were found and handled**, one of which had been published on all four mirrors for 48 days.
- **Test suites that could not fail were made falsifiable** — 16 HelixQA banks covering 72 steps declared no assertions at all and scored PASS against a dead target; 238 steps now carry explicit expectations.

Fixed (revision 3 range)

Revision 5 disambiguation. “Fixed” in this table’s heading and in the per-row **HXC-NNN** bold labels below describes the *commit* — a bug-fix landed in source — never a tracker-item closure claim. This distinction was not explicit enough in revisions 1–4: several rows below (HXC-229, HXC-233, HXC-235) describe genuine source fixes, but the corresponding `docs/workable_items.db` tracker items are **not** closed (status Queued) and this document does not close them either. See “Revision 5 — HXC-229/233/235 tracker-vs-runtime-state reconciliation” for the full, evidence-sourced account of what is and is not verified for

each, and why reading a row here as “this tracker item is Fixed” would be a §11.4 documentation-layer overclaim.

SHA	Repo	Summary
15f00801	main	HXC-233 — completions were returning HTTP 500 “all providers exhausted” on every request. LocalRPCPort defaults to 50052 and the live process had the env var unset, so the gateway dialled a port no service listens on while helixllm-coder.service served the model on 18434. Fixed with two Environment= lines placed <i>before</i> EnvironmentFile= so an operator .env can still override. Verified: :50052 connection refused → after fix, HTTPS 200 naming the real .gguf. Findable only because HXC-235’s deploy an hour earlier replaced a misleading TLS error with an honest one.
fee7e1bc	main	HXC-233 guard — scripts/testing/guard_hxc233_completion_path_live.sh, asserting a <i>real</i> assistant message and a model id, not a status code. Its load-bearing proof is Mutation B: a stub returning HTTP 200 with an error envelope makes the guard exit 1, which a status-only assertion would have passed.
70cb8821	main	HXC-244 RED baseline — establishes as fact that cmd/helixllm/main.go:148 builds health.NewChecker() and registers nothing; of the 41 .Register() call sites in the tree, zero are on the health checker. The guard deliberately asserts the component list , not status=="healthy" — that assertion passed on the broken build for the wrong reason. Two defects in the guard itself were caught by its own golden-good/golden-bad fixtures (§11.4.107(10)): an invalid f-string made the analyzer crash on every input, and the crash’s exit 1 read as “defect found”, so revision 1 reported RED-confirmed against a <i>healthy</i> fixture. Fixed structurally — analyzer findings start at exit 2, and any exit

SHA	Repo	Summary
		outside $\{0\} \cup \{2..6\}$ is a broken instrument and hard-fails in both polarities.
262a44ad	main	HXC-244 closed at the RUNTIME layer. The source fix (helix_llm 8260cf8, 969 insertions across 9 files with tests) had merged but never reached production: the running binary's mtime predated it and go tool nm \ grep -c registerHealthChecks returned 0. Closed by rebuild + redeploy of the gateway only — §9.2 backup first, artifact verified <i>before</i> install (registerHealthChecks 6, Ping values 4), NRestarts stayed 0. After: llm_providers, kv_cache_redis, vector_store_qdrant, llms_verifier, each carrying a measured duration — the evidence of a real round-trip that distinguishes this from a hardcoded list.
f2e12570	main	HXC-235 deployed — the deployed binary was 15 days stale , containing semantic_embeddings zero times, so four helix_llm commits (HXC-233's fix among them) were not live either. The process-start time had suggested a 13-hour gap; strings on /proc/<pid>/exe gave the real answer in one command. Post-deploy the key appears with value false, which is <i>correct</i> (HELIX_EMBEDDING_PROVIDER is unset, so the pipeline genuinely is the HashEmbedder) — the defect was that a caller could not tell.
724d2bb0	main	HXC-229 — the gateway ran in its web framework's debug mode under systemd. Environment=GIN_MODE=release added to the unit; verified on the running process via /proc/<pid>/environ, with 0 debug lines in a window provably containing a real startup.
e879702c	main	265 data races eliminated in applications / desktop. Root cause: a code comment asserting (*widget.Entry).SetText is goroutine-safe, which the toolkit's own thread.go has contradicted verbatim since

SHA	Repo	Summary
		v2.6. RED 265 WARNING: DATA RACE exit 1 → GREEN 0 races exit 0 at -count=3.
905a0b0a	main	A guaranteed StreamWithTools deadlock (not a race): GenerateStream was inlined at both call sites writing into a 100-buffered channel nobody drained. Plus six provider files with unguarded sends leaking a goroutine and an open HTTP body on client disconnect.
dd3c0c3b	main	Third confirmed “unprioritized select” race — autoSaveLoop could fire one more SaveAll() after DisableAutoSave() had closed stop. 2488/5000 forced-race hits prefix, 0/5000 post-fix. (<i>Documented in revision 2 but filed under the wrong range — see “Independent review”.</i>)
0bcd2e98 + 6d8a4920	main	HXC-228 — infra “started” now means “serving” rather than “created”; verified by a full-target cold boot with zero restarts.
11861996	main	HXC-168 / CONST-042 — a database password carried as a literal in shipped setup and container files published to all four mirrors. Reachability was <i>established, not assumed</i> : two compose files published Postgres on all interfaces, and the autoboot one boots by default. Explicitly does not close the item — rotation is outstanding.
be599c10 + ce5ecc04	main	HXC-167 — transcript redaction was fail-open and allowlist-only; three further fail-opens found, one of them <i>inside the guard itself</i> .
e6118f99	main	Dropped inverted docs_chain derive edges that would have destroyed 156 tracked items .

Added / Changed (revision 3 range)

SHA	Repo	Summary
ae218ee3	main	

SHA	Repo	Summary
		Cascades §11.4.236 (QA-deploy-readiness gate — no manual-QA hand-off until the mandated validation produced a candidate-fingerprinted PASS verdict; a blocker must bind to a seam, never prose) and §11.4.237 (mandatory exhaustive context-and-spirit-aware translation review — accuracy necessary, never sufficient) into all six consumer carriers, closing a measured §11.4.157 lockstep gap (canon at .237, consumers at .235). Verified byte-identical across all six (md5 df2cb867668b82e66e2d25235cd1dec5).
74f24528	main	Cascades §11.4.238 (automated QA must be the DISCOVERER: manual QA finds nothing new, and anything it finds is a coverage escape) into all six carriers and bumps the constitution pin to 177f2b0 in the same commit per CONST-049 step 7, after verifying the head published on all 8 constitution remotes. §11.4.238's forensic anchor is <i>this project</i> : every defect closed that day — the health endpoint, the dead completion port, the two unfetchable pins, both hardware-profiler defects — was found by an agent reading source or probing by hand, while HelixQA reported green throughout.
f0dd7069	main	Bumps four submodule pointers so a fresh clone actually gets the fixes — every fix lived in a submodule while the parent still pinned a commit predating it. Every old pointer verified an ancestor of its new head (FF-safe ×4, no force), every head verified published. Notable §11.4.6 datum: <code>git rev-list --count HEAD..@{u}</code> reported behind=0 <i>before</i> the fetch; after <code>fetch --all</code> , behind=3 — remote state is not knowable without fetching.

SHA	Repo	Summary
204bc7b4	main	Two third-party pins recorded on main that no clone can fetch . The dispatching premise was wrong and the correction is the finding: 7 of 9 suspected pins were fine. The real defect is that signoz and skyvern were explicitly listed as deliberately-not-recorded, then recorded anyway by an Auto-commit. Two independent oracles agree neither exists upstream (GitHub REST 422; live upload-pack “not our ref”).
33ee0670	main	HXC-243 — of 129 HelixQA banks / 243 http steps, 16 banks / 72 steps declared no assertions at all and the executor compares a field only when declared, so they scored PASS on any response. Baseline measured: 16/16 banks PASS against a dead target, exit 0. Now 238 steps carry an explicit expectation, 5 an explicit skip+reason, 0 assertion-free. Key finding: expect_status alone would not have fixed it — the wrong port answers 200 with a JSON-RPC error envelope.
d8e46001	main	A HelixQA agent proved a PASS-bluff in our own test banks by construction — it pointed a bank at the wrong service (a completion path already proven dead with HTTP 503) and wrote its thesis before running. Both cases reported PASS.
9f8fdd27	main	HXC-227 — a live provider key published in a tracked design doc, committed 2026-06-19 and present on all four mirrors for 48 days , under a heading reading “already configured”. The discovering agent’s handling is recorded as correct: it recognised that exporting the doc would copy the key into two <i>new</i> tracked files, removed the artifacts and quarantined the source rather than

SHA	Repo	Summary
		attempting a scrub. Only rotation withdraws it.
09a086a6	main	HXC-187 / D-7 — renames the thin root module to <code>dev.helix.code/meta</code> so the root and inner modules stop claiming the same identity.
1dab3e06 + 40c8d5ce	main	Tune test parallelism from the binding cgroup quota , not <code>nproc</code> . Paired with 918f969c, which explicitly corrects its own earlier root cause (“GOMAXPROCS does not explain the agent stalls”).
732320c8	main	Untracks 104 MB of vendored <code>node_modules</code> from the agent repository.
ee39ea30, 9f68ed04, cf4eb22a, e9a1ecb6, 69772e56	main	§11.4.65 sibling-export backfill (101 root docs, 73 inner docs, 62 under <code>docs/**</code> , script companions, 4 missing <code>.pdf</code>). These account for the bulk of the range’s 458k inserted lines.

Tracker / workable-items (revision 3 range)

SHA	Repo	Summary
ac9e9e4d, 38d8f641	main	Closed verified items in two passes; High severity 8 → 4, Critical 0. 38d8f641 also caught that the shipped <code>workable-items</code> binary was ten days older than the invariant its own source defines , so validate reported OK on records the source rejects.
8214bd93, e84e5b4c	main	HXC-217 — 15 closure records described their proof in prose instead of pointing at it, which is mechanically

SHA	Repo	Summary
		indistinguishable from having no proof. Narratives preserved verbatim in a new <code>item_history.note</code> column; a resolvability validator added that refuses a closure whose stated location does not resolve.
679e2e2e + 825e1c3c	main	HXC-245 filed, then its own filed root cause retracted as wrong, refuted by the code — the correction is the artifact worth reading.
79939789	main	HXC-246 — the agent test suite reports 74 failures, filed with classification <i>deliberately open</i> . Three signatures, none yet proven a product defect and none yet cleared: 22 test files hardcode a cache port that is closed on this host (the container publishes it elsewhere); a cluster failing in 0.00s while the service under test is provably healthy; wall-clock assertions overshooting ~26% under measured contention. The commit withdraws one of its own guesses explicitly.
b48d235e	main	HXC-247 — two owned, individually-correct components both claim port 8100. The agent's

SHA	Repo	Summary
		port allocator assigns it 8100 while its shipped config still pins 7061, so the verifier takes 8100 first; 82 test files address 8100 expecting the agent, receive the verifier's 404, and report the agent broken while it is healthy. Root cause of HXC-246's 0.00s cluster. Contains a self-correction retracting an earlier "8090" claim with measurements.
d706f8cc	main	HXC-248 — integration-test cleanup decides whether to tear down the stack by checking only that <i>something</i> answers on port 8100. That something is the verifier, not the agent, so teardown is skipped and the live platform survives by coincidence; stop or move the verifier and the same check would tear down a running deployment. The log line it prints is also false.
1f177495	main	HXC-244 filed — the defect was found, fixed, deployed and verified, but no tracker entry was ever created; HXC-243 and HXC-245 existed and 244 was a genuine gap. Filed directly as Fixed (→ Fixed.md) against

SHA	Repo	Summary
		the runtime evidence above.
7bd80ff6	main	Swept all 48 non-terminal items against <code>git log --all -i --grep=<ID>; closed HXC-199 and HXC-217 on re-verified evidence and deliberately left 46 open. Records a systemic finding: <code>verify-all-constitution-rules.sh</code> invokes 21 <code>scripts/gates/entries</code> and zero of the three <code>scripts/testing/guard_hxc*.sh</code> guards written this cycle — a guard nothing runs cannot fail (§11.4.135).</code>
5725ee06	main	Normalises 16 severity values that held entire triage paragraphs where a closed-set token belongs. 10 were safe to reduce because the rationale already rendered from <code>body_md</code> ; 6 were not — the column was their only copy, so the rationale was first written into <code>body_md</code> and only then reduced. Prose 16 → 0, valid 230 → 246, with all 16 rationales proven to survive a full regeneration.

Revision 4 — the 5725ee06..84853455 range (6 main commits) + submodules/helix_agent's full 66a3c1c6..HEAD range (24 commits, 2026-08-09 → 2026-08-11)

Everything in this section was derived on 2026-08-10/11 by reading `git log` and `git show` directly against both repositories, cross-checked against `docs/workable_items.db` (§11.4.93/§11.4.95 — the tracker's single source of truth) for the status of every tracker item cited, rather than inferring status from commit-message prose (§11.4.6). This section also fully characterises `submodules/helix_agent`'s entire 66a3c1c6..HEAD range, which revision 3 recorded only as a pin/checkout mismatch (Known gap 14) and explicitly declined to enter.

Summary — what changed for a user in this range

Grouped by theme (§11.4.91), not by commit order:

- **A whole class of false “HelixAgent is broken” test failures is fixed at the root, not patched symptom-by-symptom.** Roughly 74 + 20 + 13 + more integration-test failures across five separate packages traced back to variations of one cause: a test found *something* answering on a well-known port and treated that as proof HelixAgent was there, when the something was frequently a different service (LLMsVerifier squatting on :8100, Weaviate squatting on :50051, a stale `configs/development.yaml` port) or a password-protected Redis rejecting an unauthenticated command. Every guard in this range now verifies *identity* — a real `{"service": "helixagent"}` payload, a completed Redis AUTH+PING handshake — never mere reachability. None of this is a HelixAgent product regression; all of it was the test suite blaming the product for infrastructure it never actually contacted.
- **HelixAgent's gRPC server can now start on a host that is already running HelixAgent's own infrastructure — landed in source, verification on a rebuilt/redeployed artifact pending.** It previously hardcoded :50051, the exact port this project's own Weaviate container publishes, so on a normal deployment the gRPC listener could not bind at all; a second-order bug then made the integration test client silently dial the live Weaviate instance instead and report 20 failures against HelixAgent, which was never actually reached. Fixed by registering the service in the internal port registry (:8112 by default) and re-deriving every place that names the port — the server, the startup banner, the docs, the Kubernetes manifests, and a protocol-compliance challenge script — from that one source, across seven commits landed over two days as review rounds surfaced further instances of the same class (the banner not tracking the listener; the k8s manifests naming a

dead env var and an unbound port; a challenge script recording success on every branch regardless of outcome).

- **Generated CLI-agent configuration files pointed 46 of 48 downstream AI coding assistants at the wrong service — landed in source, one known-unwired residual documented below, so treat as not confirmed complete.** `helixagent -generate-agent-config=<name>` (and the “generate all agents” / Crush variants) wrote `localhost:8100` into every emitted config regardless of the port the server actually bound, from a second, independent hardcoded source beyond the one an earlier commit in this same range had already fixed for 2 of the 48 generators. An independent review returned this fix for further work twice, surfacing an unenforceable dial host that could resolve to the unconnectable wildcard address `0.0.0.0` (reachable today via a committed Docker Compose file, no mutation needed), a “reachability” probe that misclassified several real wrong-service responses as “nothing is here”, and inconsistent handling of malformed port values across the four generators — including, on a first attempt at the fix itself, the ports 0, -1, and 65536, all still silently accepted. **This documentation pass independently found one further call site the fix does not touch** — see “Known gaps”.
- **The scripts operators run to certify “the platform’s tests passed” could report that without running a single test.** `run_complete_validation.sh` and `orchestrate_full_test.sh` accepted a 404 from the wrong service as “ready” (a plain `curl` without `-f` treats that as success), and a shell arithmetic bug (`grep -c ... || echo "0"`) turned zero-count summary fields into the two-character string `"0 0"`, which either prints garbage or aborts a downstream numeric comparison whose failure then reads as “tests ran”. Both scripts now discover the server’s real address by asking the kernel which port the process they started actually bound, verify its identity, and treat zero-tests-executed as a hard failure rather than a silent success.
- **`go.mod/go.sum` were quietly out of tidy** since a dependency bump two work sessions earlier — 30 stale superseded-version hashes plus 2 structurally incomplete entries, invisible to `go build/go vet` but a real, deterministic `go mod tidy -diff` failure since the measured baseline.
- **The project’s own workable-items tracker had drifted from its database of record, and the drift was hiding a governance-gate violation.** `docs/Issues.md` was 18 open items and 3 days stale relative to `docs/workable_items.db`; regenerating it also surfaced and fixed two items that had never been indexed at all (a code path that bypassed the sanctioned writer), plus an obsolete-item record missing its required detail line, hidden because the stale document sorted it out of view. Three standing regression guards written earlier in the project’s history had never actually been invoked by anything — a guard that is registered but never run cannot fail — and are now wired into the release-gate sweep (32 gates, up from 29), after three independent review rounds each found and the author fixed further

false-positive and false-negative failure modes in the guards themselves before they were trusted.

- **An independent review of the tracker-repair and guard-wiring work returned 8 findings (1 HIGH, 3 MEDIUM, 4 LOW), all individually dispositioned rather than bundled into a single “addressed” claim.** The HIGH finding was itself a live data-destruction trap: the tracker’s own sync-consistency gate, on detecting drift, printed a specific repair command that — in the exact drift direction this range’s own repair work had just hit — would have overwritten the 46-item database with a stale 36-item document, permanently deleting 10 items and resurrecting 8 already closed ones. The message no longer asserts a direction it cannot prove; it states what was detected and lets the operator choose.

Fixed — main repo (revision 4 range)

SHA	Repo	Summary
04f88a23	main	docs/Issues.md had drifted 18 open items behind docs/workable_items.db (last regenerated 3 days earlier) and was hiding a governance-gate violation: HXC-230, marked Obsolete, was missing its required **Obsolete-Details:** line, invisible because the stale document sorted a different item above it. Root-caused two independent causes rather than assuming one: HXC-247 and HXC-248 had been filed through a raw INSERT that skipped the row the renderer needs to see an item at all, and the document itself was simply stale. Repaired by regenerating from the database — the opposite

SHA	Repo	Summary
		of what the project's own sync-gate failure message recommended at the time, which is HXC-252, fixed later in this same range by f10c9a1e.
be5d56be	main	Three standing regression guards (guard_hxc229, guard_hxc233, guard_hxc244) existed on disk but were invoked by nothing anywhere in the release sweep — HXC-244 had been closed against a guard that had never once run in any suite. Wired all three in as new gates G30–G32. An independent review returned NO-GO on the first attempt and found three further blocking holes (a guard that read “cannot reach a dead HTTP proxy” as “service absent”; a guard that read a crash-looped systemd unit as “not installed”; an unfalsifiable prose claim with no re-runnable evidence behind it) plus four warnings, all fixed before the guards were accepted — replacing prose assertions with a 22-case, then 23-case falsification battery that builds its own live test

SHA	Repo	Summary
		subjects (HTTP stubs, transient systemd units, a deliberately crashed analyzer) and proves each PASS/FAIL/SKIP boundary rather than asserting it.
69573772	main	Seven findings that existed only in earlier work-session notes were formally filed (HXC-249 through HXC-255) rather than left to be lost, each independently re-verified against the live tree before filing. HXC-247 (two services both claiming port 8100) was updated in place once its deployment intent was confirmed from a service-unit file, rather than duplicated as a new item.
b947276f	main	Two further findings that had only been reported in prose in the prior review round were filed as tracked items: HXC-256 (the tracker's export command silently writes to the process's current working directory when --out-dir is omitted — the two sanctioned wrapper scripts always pass it explicitly, so exposure is bounded to direct ad-hoc invocation) and HXC-257 (a sync-gate script's diagnostic

SHA	Repo	Summary
		message calls its comparison input “the committed database” when it is actually a copy of the uncommitted working-tree file — reproduced live: the gate printed “committed DB validates” while the database held seven items that had not yet been committed anywhere).
f10c9a1e	main	An independent review of be5d56be and 04f88a23 returned GO with no blocking defect but 1 HIGH, 3 MEDIUM, and 4 LOW findings; §11.4.134 requires zero findings of any severity before a change is accepted, so all eight were individually dispositioned. HIGH (HXC-252, fixed): the sync-gate’s failure message recommended a repair direction that, in the drift direction actually measured on 2026-08-09, would have overwritten the 46-item database with a stale 36-item document. MEDIUM (all fixed): the new guards’ SKIP/FAIL boundary was missing negative-space test coverage; a guard that had been permanently reporting SKIP on the

SHA	Repo	Summary
		production host (because one of its two checks could never be proven from the available evidence window) now reports PASS-with-caveat, naming which check ran and which could not, rather than certifying nothing forever; and two of the prior commits' own quantitative claims (a "row-identical" data-safety claim, and a "22 ok" count) were found imprecise and corrected in a dedicated evidence file, since the original commit messages cannot be edited after the fact (§11.4.113). A further MEDIUM finding was root-caused rather than hand-patched and filed as a new item (HXC-258) instead: a tracker-metadata field that structurally cannot ever record the value its name promises, because its one writer only ever runs on the opposite sync direction.
84853455	main	Files the two defects the submodule-side gRPC-port work below found (HXC-260 High, HXC-261 Critical) and bumps the submodules / helix_agent gitlink to the commit containing

SHA	Repo	Summary
		both fixes — the same movement that resolves this document's Known gap 14 (pin/checkout divergence).

Fixed — submodules/helix_agent (revision 4 range, pinned into main's gitlink 94e2dcc8)

SHA	Summary
16a8a1d1	The integration suite's largest failure cluster (74 assertions, all expected 200, actual 404) was not a HelixAgent defect: LLMsVerifier was squatting on the default port :8100 while the real HelixAgent answered correctly on :7061. The guards used <code>if err != nil { t.Skip() }</code> , which only detects an <i>empty</i> port, so a live-but-wrong responder passed straight through undetected. Fixed by resolving the target by identity — a real <code>{"service": "helixagent"}</code> health check — not by port.
d13132e3	A generator test hardcoded one developer's absolute local filesystem path as its working directory, so it could not run on any other checkout — a <code>chdir: no such file or directory</code> failure carrying zero information about the product. Fixed to resolve the repository root at runtime.
6fbf6282	The Redis health check and the code it guards resolved <i>different</i> fallback ports from the same environment variable (:6379 vs :8110), so on a host with no explicit Redis configuration the guard reported “available” while the guarded code dialled a closed port and failed. Five

SHA	Summary
	different fallback values for the same setting existed across the module; consolidated into one resolver. The availability probe also stopped treating any TCP response as “usable” — it now completes a real AUTH+PING handshake, since a password-protected Redis instance was previously reported available while every real command against it would fail.
fe183ce4	The AUTH+PING handshake 6fbf6282 added had no test coverage of the case it exists to catch — proven by weakening the check and watching three packages stay green while the gate accepted a password-rejecting Redis endpoint as usable. Added the missing positive/negative fixture pairs.
34df0832	Documents (does not silently reconcile) three call sites that resolve their own Redis endpoint independently of 6fbf6282’s unified resolver — reconciling one of the three the obvious way would re-open the exact defect 6fbf6282 just closed. Adds a ledger test that fails if any of the three is quietly migrated without the ledger being updated to match, so a stale ledger cannot go unnoticed.
1796a588	The identity gate guarding roughly 30 integration test call sites had zero tests of its own — a one-character typo in its service-name check made every guarded test silently skip while go test still reported success. Added the missing self-test, and split a failure message that had been guessing a single cause (“another process is bound to this port”) for four structurally different outcomes, three

SHA	Summary
	of which were genuine HelixAgent regressions being misreported as an unrelated port conflict.
32d2e8cb	Follow-up review finding: every test exercising the “strict identity required” environment variable read that variable’s name from the same constant the product code reads, so renaming the constant would silently break strict mode everywhere while the test suite stayed green — the same self-referential-loop pattern already closed for the service-name check by 1796a588. Closed with a test that writes the documented variable name out longhand and drives the real resolver through it.
526693cd	Documents the HELIXAGENT_REQUIRED strict-mode flag in .env.example (commented out; changes no default behaviour) and records, with evidence, why no existing make target is currently a safe place to enable it by default — the obvious candidate’s own infrastructure step never starts the HelixAgent server it would be asserting against.
159dd839	The shipped Containerfile for the MCP-servers image could not be built at all — a prior commit relocated the directory it COPY’d without updating the COPY instruction. Fixed with an opt-in build stage; verified end-to-end that the image now builds by default and its seven active MCP servers start correctly.
816f37c2	All 71 container-backed MCP server configs disagreed with themselves — a URL field derived from the current port table, and a separately hand-

SHA	Summary
	written port number from a superseded numbering scheme. A caller reading the port number reached nothing; a caller reading the URL reached the container. Fixed by deriving both fields from one lookup.
70e28ead	The OpenCode CLI provider blocked for its full configured timeout — measured up to 60s against a 6s budget — even after the model had already answered in milliseconds, then discarded the answer in favour of a timeout error, because it waited for the whole child process tree to exit rather than returning once the conversational turn closed. Also fixed the timeout error message, which always reported the provider’s own configured timeout even when a shorter caller-supplied deadline was the one that actually fired.
bc26122f	The OpenCode config generator emitted a legacy, no-longer-canonical model identifier and provider name (Helix Agent, helix-debate), so an explicit request for the multi-model debate ensemble sent through OpenCode could be silently downgraded to a single provider. Traced to a specific historical regression commit and fixed at the generator; the server still accepts both spellings so already-deployed configs keep working.
b88dadd9	go.sum had been out of tidy since a dependency bump two work sessions earlier: 30 stale superseded-version hash lines plus 2 entries missing their required /go.mod hash. Not shown to break today’s build, but confirmed structurally incomplete and

SHA	Summary
	deterministically un-tidy; fixed with <code>go mod tidy</code> .
1a55c8aa	The two scripts operators run to certify “the platform’s integration tests passed” could report exactly that without contacting the real server or running a single test — a squatting service’s 404 satisfied a <code>curl</code> call made without <code>-f</code> , and the guarded tests simply skipped against the wrong port, so <code>go test</code> exited 0 on 3 skips / 0 passes / 0 failures while the script logged success. Fixed to discover the real server address by asking the kernel which port the started process actually bound, verify its identity, and treat zero-tests-executed as a hard failure.
604a2f49	The <code>-generate-opencode-config / -generate-crush-config</code> generators wrote a base URL for port <code>:8100</code> while the server itself, by default, binds <code>:7061</code> — handing real users a config pointing at whichever different service happened to be listening on 8100. Fixed by resolving the port through the same configuration path the server itself binds from.
886f72c0	The validation-readiness helper 1a55c8aa added had no self-test: three independent one-line mutations each silently inverted a user-visible pass/fail verdict with nothing in the repository failing. Added a golden-good/golden-bad self-test pair covering all three, each proven to detect its own corresponding mutation.
ad3b5590	The exact <code>grep -c ... \ \ echo "0"</code> shell arithmetic bug that 1a55c8aa’s own commit message

SHA	Summary
	documented as a footgun was still live at four other call sites in the two runner scripts it fixed, including the very summary block that commit was about — rendering zero-count fields as the two-token string "0 0". Fixed at all four remaining sites.
d2d70206	HelixAgent's own gRPC server hardcoded :50051 and refused to start whenever this project's own Weaviate container (which publishes that exact port) was already running — HelixAgent's gRPC service could not coexist with HelixAgent's own infrastructure. With the server down, the integration test client dialled the same literal port, completed a real handshake with Weaviate instead, and reported 20 failures against HelixAgent, which was never actually reached. Fixed by registering the service in the internal port registry (:8112 by default, operator-overridable) and resolving both server and test client from it.
ec95a277	d2d70206 moved the gRPC listener to :8112 but left the startup log line telling every reader to dial :50051 — the exact wrong address that reaches Weaviate instead. Fixed so the banner reports the address actually bound rather than a re-stated literal; added a guard, since this exact class of drift had no test coverage before, which is why it survived the original fix, its review, and a green suite.
14a779fe	Ten further user-facing artifacts (READMEs, a website doc page, a CLAUDE.md, generated architecture diagrams, an env-var table) still told readers to dial the pre-d2d70206 port

SHA	Summary
	:50051. Corrected all ten. Separately discovered that the override variable operators would reasonably expect (GRPC_PORT) is read by no Go code anywhere, while the one that actually works (HELIXAGENT_PORT_GRPC) had never appeared in any doc, .yaml, or .env file before this commit.
d17cee81	The two generators 604a2f49 fixed were only 2 of 48 CLI-agent config generators; the other 46 emitted localhost:8100 from a second, independent hardcode source (the shared MCP-server URL list). An independent review found three further defects before accepting the fix: an unenforced dial host that could resolve to the unconnectable wildcard 0.0.0.0 (reachable via a committed Docker Compose file, no mutation needed); a “reachability” probe misclassifying several real wrong-service responses as “nothing here”; and inconsistent malformed-port handling across the four generators, including port values 0, -1, and 65536 all still silently accepted on the first attempt. See “Known gaps” — one further call site was not touched by this fix.
76b25796	A gRPC protocol-compliance challenge script recorded every one of its four checks as "true" whether the target answered or not, and probed a fourth, never-used port spelling (:7062) that no HelixAgent process binds — certifying “gRPC protocol healthy” on every run without ever contacting a gRPC server (filed as HXC-261, rated Critical — this was actively producing false-green results, not a merely latent gap). Fixed to resolve the registry port,

SHA	Summary
	fail closed with an explicit SKIP-with-reason when nothing is listening, and never report success for a check that was never performed.
94e2dcc8	The project's Kubernetes manifests (Deployment, both Services, the NetworkPolicy, and a ConfigMap) published the gRPC port as 9090 behind a GRPC_PORT environment variable no Go code in the project reads, so editing that variable in a deployment would have had zero effect (filed as HXC-260). Corrected all four manifests to the registry's :8112. Honestly documents that this makes the <i>declared</i> endpoint correct but still inert — no container in the current Deployment actually runs the gRPC server binary — so this is address-correctness, not a new capability.

Tracker / workable-items (revision 4 range)

Status read directly from `docs/workable_items.db` at documentation time, not inferred from commit-message prose (§11.4.6). A commit landing a source-level fix does **not** by itself close the corresponding tracker item — several items below remain Queued even though the code change they describe has already merged, because closing requires a separate `workable-items close` step with its own captured evidence, which this documentation pass did not perform:

Item	Type / Severity	Status (per DB)	Note
HXC-244	Bug / High	Fixed	Covered in revision 3; re-confirmed here.
HXC-249	Bug / Critical	Fixed	Validation runner false-PASS, closed this range.
HXC-252	Bug / High	Fixed	Sync-gate data-destruction-risk message, closed this range.

Item	Type / Severity	Status (per DB)	Note
HXC-253	Bug / Medium	Fixed	Standing guards wired (G30–G32), closed this range.
HXC-199, HXC-217	Bug	Fixed	Closed in 7bd80ff6 (revision 3); re-confirmed here.
HXC-229, HXC-233, HXC-235	Bug	Queued — see the dedicated reconciliation below	Source-level fixes described in revision 3 (724d2bb0, 15f00801, f2e12570); all three currently have genuine, tracked, captured runtime evidence, and are not simply “verification pending” — see “Revision 5 — HXC-229/233/235 tracker-vs-runtime-state reconciliation” for what each item’s own review/diary trail actually shows. Do not read revision 3’s “Fixed” table headers as tracker-closure claims — the DB is the status of record.
HXC-247	Bug / High	Queued, updated in place	Deployment intent for the port-8100

Item	Type / Severity	Status (per DB)	Note
			collision confirmed; not yet resolved.
HXC-248, HXC-250, HXC-251, HXC-254, HXC-255, HXC-256, HXC-257, HXC-258, HXC-259, HXC-260, HXC-261	various	Queued	Newly filed or updated this range; none closed by this document.

Total tracker size at documentation time: **455 items** (docs/workable_items.db), up from the 453 the task's own briefing cited — 2 more (HXC-260, HXC-261) were filed by 84853455 after that briefing was written, which is itself evidence of the continuous-parallel-work pattern this document keeps warning readers about (§11.4.103/§11.4.126). Status breakdown: 216 Fixed, 95 Implemented, 84 Completed, 54 Queued, 4 Obsolete, 2 Operator-blocked.

§11.4.151 release-tag prefix compliance

Checked directly against `git tag -l` in both repositories, not assumed:

- **.env HELIX_RELEASE_PREFIX=helix-code** resolves the prefix for both the main repo and every owned submodule per §11.4.151's deterministic resolution order.
- **Main repo** carries four prefixed tags (helix-code-1.0.0-dev-0.0.1, helix-code-1.1.0-dev-0.0.1/.2/.3) — all §11.4.151-compliant — **plus two legacy unprefixed tags this task's briefing did not mention: helixcode-v1.0.0 and helixcode-v1.1.0**, predating the project's adoption of the helix-code- prefix scheme.
- **submodules/helix_agent** carries one prefixed tag (helix-code-1.0.0-dev-0.0.1) and one legacy unprefixed tag, **helixcode-v1.1.0**, as the task's briefing states.
- **Finding:** the legacy-tag condition is **not unique to the submodule** — it exists on the main repo too, in the same helixcode-v naming style, at the same two version points (v1.0.0, v1.1.0). This is additional information beyond what the task asked about, reported because it bears directly on whether the fleet is §11.4.151-compliant as a whole.
- **Is this a violation requiring remediation?** §11.4.151 requires every *newly created* release tag/version to carry the resolved prefix, identical across the

main repo and every owned submodule in one release. It does not, on its own text, retroactively forbid tags created before the mandate existed — the four `helixcode-v1.0.0/helixcode-v1.1.0` tags across both repos predate the prefixed scheme (all four prefixed tags are dated after them: `helix-code-1.0.0-dev-0.0.1` is the earliest prefixed tag anywhere in this fleet).

This document's recommendation — not a settled disposition, since retagging/deleting a published ref is a destructive operation only the operator can authorize — **is to treat this as a *historical-artifact condition***, not an active violation, conditional on two things holding, both of which do hold today: (a) no *new* tag has been or will be created in the legacy unprefixed form, and (b) the legacy tags are not being used as the current release pointer anywhere (they are not — `helix-code-1.1.0-dev-0.0.3` / submodule's `helix-code-1.0.0-dev-0.0.1` are the current references).

The operator may reasonably disagree — e.g. if a policy requires every non-conforming ref removed regardless of age — and that disagreement would not make this document's factual findings (the tag list, their dates, and the ordering relative to the prefixed tags) wrong, only the recommended disposition.

- **Recommended remediation (report + recommendation only — nothing retagged or deleted by this task, per its hard constraints and per §11.4.113/CONST-043's absolute no-force-push-or-destructive-history-op posture, which extends to tag deletion without explicit in-conversation operator authorization; the numbered steps below are what this document recommends, awaiting the operator's actual decision, not a record of action already taken or endorsed):**

1. **Do not delete or move the four legacy tags without an explicit operator decision to do so.** Deleting a published tag is a destructive, history-altering operation on refs other consumers may already depend on; it requires explicit operator authorization per operation (§11.4.90/§9.2), which is out of scope for a documentation-only task and was not sought — this document takes no position on whether that authorization should be granted, only that it has not been.
2. **Document them as superseded**, exactly as this section now does, so a future reader (human or gate) does not mistake `helixcode-v1.1.0` for a currently-maintained release line, pending whatever disposition the operator ultimately chooses.
3. **Never create a new tag in the unprefixed form.** This is already true in practice — every tag created since `helix-code-1.0.0-dev-0.0.1` in both repositories uses the resolved prefix — and this document's own “no new tag” constraint keeps it true through this task.
4. If the project later wants a mechanical guard against regression, the natural implementation is the constitution's own recommended CM-RELEASE-PREFIX-NAMING gate (§11.4.151) scoped to *new* tag creation (e.g. a pre-push or tag-creation hook), not a retroactive rewrite of tag

history. That gate does not exist in this repository today (not verified further — out of this task’s scope to add one).

Revision 5 — HXC-229/233/235 tracker-vs-runtime-state reconciliation (coordinator-directed, 2026-08-10/11)

Revision 4 flagged, correctly, that revision 3’s “Fixed” table headers for HXC-229/233/235 could be misread as tracker-closure claims when `docs/workable_items.db` shows all three Queued. This section resolves that inconsistency the way it must be resolved — from evidence, per item, not by assumption — after the coordinator directed a closer look rather than accepting the generic “verification pending” phrasing revision 4 first reached for.

Method. For each item: (1) confirmed the source fix landed, via the SHA already cited in revision 3; (2) checked whether a registered, falsifiable guard exists, by reading the guard script/test file directly, not by trusting a commit message’s claim about it (§11.4.226); (3) ran what could safely be run read-only against the live host to check current state; (4) read `docs/workable_items.db`’s `item_history` (event log) and `test_diary` (per-run test log, schema-enforced: CHECK (result <> 'PASS' OR evidence_path IS NOT NULL) — a PASS row cannot exist without a captured evidence path) tables directly via `sqlite3 -readonly`, read-only, the same access already used throughout this document; (5) read every `docs/qa/evidence` directory these tables point at and confirmed each is `git ls-files-tracked`, not merely present on disk. No item’s DB status was changed by this process, per this task’s explicit instruction — this section reports; the coordinator routes.

Finding, stated once because it applies to all three: the actual state is more evidenced than either of the two possibilities this investigation was framed around. None of the three fits “source fix landed, runtime evidence never captured, so Queued is simply correct and the changelog should say ‘verification pending’” — genuine runtime evidence exists, is captured, and is tracked in git, for all three. Nor is `docs/workable_items.db` simply “stale and should say Fixed” — in two of the three cases, the tracker’s *own* independent-review process explicitly examined the item and explicitly declined to close it, for a specific, evidenced reason; what changed since is that the specific reason cited has since been addressed by later, uncoordinated work, without anyone re-running the review or the closure step that would act on that. Describing these three the same way as the CLI-config and gRPC port fan-outs (“landed in source, verification pending”) would **understate** real, current, git-tracked evidence — a different but equally real documentation-layer inaccuracy from the one flagged. The precise, evidenced state of each follows.

HXC-229 — gateway release-mode

- **Source fix:** 724d2bb0 (main repo) — adds `Environment=GIN_MODE=release` to the gateway's systemd unit.
- **Registered guard:** `scripts/testing/guard_hxc229_gateway_release_mode.sh` exists, reads `/proc/<pid>/environ` on the live gateway process (not the unit file — the guard's own header explains why config-only assertion is rejected as a §11.4 bluff), and is wired into the release-gate sweep as **G30** (be5d56be, this range).
- **Last recorded tracker verdict (test_diary entry 36, 2026-08-08T08:32:34Z, docs/workable_items.db): PASS** — “Live runtime validation PASSES with a real runtime signature; closure still blocked on the missing §11.4.135 standing guard” — evidence path `docs/qa/independent_review_hxc229_234_235_20260808T132900Z/hxc229_live_journal_since_restart.txt` (git-tracked, confirmed). This diary row is itself the independent review's verdict (commit c8171122, “independent review of HXC-229/234/235 — all three NOT-CLOSED, three distinct layer failures” — HXC-229's stated, singular reason: **“FIX VERIFIED LIVE; NO STANDING GUARD.”**)
- **What has changed since that verdict:** the missing guard — the *sole* cited blocker — was added and wired as G30 by be5d56be, later in this same unpushed range.
- **Re-verified live by this documentation pass** (read-only, `bash scripts/testing/guard_hxc229_gateway_release_mode.sh`, 2026-08-10/11):
PASS-with-caveat (HXC-229): `pid=107268` carries `GIN_MODE=release`, read from the live process's own `/proc/107268/environ` — the primary invariant is **ENFORCED** and holds. The same PID (107268) also appears in this range's own `docs/qa/r4_review_remediation_20260810T191500Z/EVIDENCE.md` (already cited in this document's revision 4 tables), meaning the live gateway has not restarted between that evidence capture and this one — the PASS is not a one-off fluke, it is the same long-running process answering consistently across two independent checks.
- **Conclusion:** the specific reason HXC-229 was blocked from closure no longer holds. No new `test_diary` entry, `item_history` closure event, or re-review has been recorded reflecting that. This reads as a **stale blocking condition**, not a stale tracker status in the blanket sense — the Queued status is an accurate record of “not yet formally closed,” it is the *reason* that has gone stale.

HXC-233 — gateway completion path

- **Source fix:** 15f00801 (main repo) — routes the gateway’s local provider at the port `helixllm-coder.service` actually serves (18434), not the unwired default (50052).
- **Registered guard:** `scripts/testing/guard_hxc233_completion_path_live.sh` exists, asserts a real assistant message and model id from a live completion (not a status code), and is wired into the sweep as **G31** (be5d56be).
- **Captured evidence:** `docs/qa/hxc233_routing_fix_20260808T140500Z/FINDING.md` (git-tracked, confirmed) — a real, live, pre/post flip: HTTP 500 {"error":{"message":"...dial tcp 127.0.0.1:50052: connection refused"}} → HTTP 200 {"choices":[{"message":{...,"content":"2 + 2 equals 4."}}], "model":"/models/Qwen3-Coder-30B-A3B-Instruct-Q4_K_M.gguf"}, captured against the real public HTTPS endpoint.
- **No test_diary entry and no independent-review commit exist for HXC-233** — `sqlite3 -readonly docs/workable_items.db "SELECT * FROM test_diary WHERE atm_id='HXC-233'"` returns zero rows, and no commit analogous to c8171122 (which covered 229/234/235, not 233) was found in `git log --all -i --grep="HXC-233"`. Unlike HXC-229 and HXC-235, this item was never routed through a closure review of any kind, blocked or otherwise — it is not that a reviewer found a reason to keep it open; no reviewer appears to have looked at closure for it at all.
- **Re-verified live by this documentation pass** (read-only, `bash scripts/testing/guard_hxc233_completion_path_live.sh`, 2026-08-10/11): GREEN (HXC-233): live completion answered "4" from model `"/models/Qwen3-Coder-30B-A3B-Instruct-Q4_K_M.gguf"` — real generation through `https://localhost:8443/v1/chat/completions`, not a status code.
- **Conclusion:** fix, guard, and captured evidence all exist and are currently live-verified, but the item has **never been reviewed for closure at all** — a gap, not a resolved-but-unactioned block like the other two.

HXC-235 — semantic-vs-hash embeddings signal

- **Source fix:** e68fcdc (submodules/helix_llm, not helix_agent) — reports which embedding pipeline actually served a request at the point of use.
- **Registered guard:** no standalone bash script exists (unlike 229/233), but two Go test files are committed and tracked in submodules/helix_llm: `internal/gateway/hxc235_embedding_semantic_signal_test.go` and `internal/knowledge/hxc235_semantic_signal_test.go` — both part of that submodule’s ordinary `go test ./... run`, so “registered” in substance

even though the implementation differs in form from 229/233's standalone scripts. The independent review below confirms these are genuinely falsifiable (mutation-tested), not merely present.

- **Last recorded tracker verdict (test_diary entry 35, 2026-08-08T08:32:21Z): FAIL** — “Independent review: source correct and guard genuine, but fix is NOT live on the deployed endpoint” — evidence path docs/qa/independent_review_hxc229_234_235_20260808T132900Z/hxc235_deployed_endpoint_response.json (git-tracked, confirmed). The same review commit (c8171122) states the reason precisely: **“CORRECT AND GENUINELY GUARDED, BUT NOT LIVE”** — §11.4.108 layers 3–4 (runtime-on-clean-target, user-visible) unsatisfied because the deployed gateway binary predated the fix by 15 days.
- **What has changed since that verdict:** docs/qa/hxc235_deploy_20260808T135200Z/FINDING.md (git-tracked, confirmed) — timestamped **2026-08-08T13:52Z, 23 minutes after** the FAIL verdict above (2026-08-08T08:32:21Z UTC ≈ 13:32 local at this host's +05:00 offset) — records exactly the redeploy the FAIL verdict was waiting on: §9.2 backup taken, only the affected unit restarted (helixllm-coder, a separate oneshot serving the 30B model, confirmed untouched), and a real pre/post flip on the live endpoint — semantic_embeddings key count 0 → 1, NRestarts=0, HXC-229 guard re-passing on the new PID (cross-confirming that guard reads the live process, not a remembered one).
- **This documentation pass could not re-run a live guard for HXC-235** (there is no standalone script to invoke, and running the submodule's go test suite was out of scope — submodules/helix_agent is explicitly off-limits, and this fix in any case lives in helix_llm, a different submodule this task was never scoped to touch). The finding above rests on the two committed, tracked evidence artifacts, not on a fresh re-run.
- **Conclusion:** same shape as HXC-229 — the specific reason cited by the last review (not live on the deployed artifact) was addressed shortly after by a redeploy with its own captured, tracked evidence, but no new test_diary entry or re-review followed to close the loop. Per the tracker's own audit trail, the *last recorded test verdict* for HXC-235 is still FAIL — technically accurate as the last recorded event, but stale relative to the redeploy that came after it.

What this means for the changelog record (this document's own correction)

- Revision 3's “Fixed” table entries for 724d2bb0, 15f00801, and f2e12570 are **left as written** — they accurately describe commits that fixed real defects, and revision 5's disambiguation note (added above the table) now

makes explicit that “Fixed” there means “this commit fixed a defect,” not “this tracker item is closed.”

- Revision 4’s tracker-status table row for these three items is corrected (see above) to stop pointing at the generic “verification pending” framing and instead point here.
- **No blanket claim of “fixed” or “verification pending” is accurate for all three at once** — HXC-229 and HXC-235 each have a *specific, resolved* blocking reason with a documented before/after; HXC-233 was never blocked by anything recorded, it was simply never reviewed for closure.
- **This document does not close HXC-229, HXC-233, or HXC-235.** Per the coordinator’s explicit instruction, no item status was changed in docs/workable_items.db by this task. The findings above — including the specific test_diary/item_history rows, the evidence paths, and the live re-verification output captured in this session — are handed back for routing to whoever owns the sanctioned workable-items close step.

Per-repo commit inventory

main (this repo) — 18 commits in 5ab97d8c..c29e1dcc (revisions 1–2 range)

*(Revision 3 correction. Revision 2 changed this figure from 18 to 19 on the grounds that dd3c0c3b had been omitted. The omission was real; the count change was not. git rev-list --count 5ab97d8c..c29e1dcc returns 18, and git merge-base --is-ancestor c29e1dcc dd3c0c3b succeeds — dd3c0c3b is dated 2026-07-28 18:13 and lies **after** c29e1dcc, i.e. in the revision 3 range, not this one. Revision 2 correctly added the missing commit to the Fixed table but attributed it to the wrong range and inflated this count to match. Restored to the measured 18; dd3c0c3b is carried in the revision 3 range above. The two figures now reconcile: 18 + 231 = 249 = git rev-list --count 5ab97d8c..HEAD.)*

dd3c0c3b fix(persistence): close unprioritized-select race in autoSaveLoop
c29e1dcc docs(qa): LIVE proof for the Anthropic wire and the i18n cwd fix
a5462e35 docs(qa): LIVE proof that stream:true reports the served model
e952f4d1 docs(qa): §11.4.83 evidence for the three non-feature commits – G
1c9fdcf2 docs(qa): §11.4.83 evidence for the Cerebras, Cohere and MCP-conc
d8c44618 docs(qa): §11.4.83 evidence for e330149f model-identity fix
925aa859 docs(governance): backfill the §11.4.182 TRUNK RULE + bump two su
e330149f fix(llm/server): report the model that ACTUALLY served the reques
578de68d docs(qa): land §11.4.83 end-user QA evidence for 7 feature commit
0a4eb8d0 docs(resume): correct live-state anchors after the end-of-session
cf26a173 chore(submodules): bump gitlinks after end-of-session sync
56f7edf7 Auto-commit
66d6fb29 fix(e2e): stop go test compiling generated challenge output (3 pa

98db7d79 feat(llmsverifier): seed real providers via consumer-owned config
d3e2c6e7 feat(systemd): add LLMsVerifier as the seventh unit, wired to the
c4115c73 feat(systemd): install and boot the whole platform – HelixCode +
1296d328 Auto-commit
565506f6 chore: bump HelixLLM to helixllm-0.2.0 (CPU inference)
f14a3f89 chore: bump HelixAgent (Hyper provider)

submodules/helix_agent — 9 commits since the previously-pinned 55ad06c6

Pinned into main's current gitlink (3268fd73) — 7 commits:

3268fd73 fix(testutil/tests): verify the server is HelixAgent, not just th
0165ab1d fix(precondition): establish container ownership from the runtime
c20215bd test(helix_agent): chaos/compliance/automation hardening + live-e
4b5f2867 style: gofmt alignment and comment spacing across helix_agent
36597718 test(search): reconcile UnknownType assertion to the reordered ty
0dfa69e1 fix(boot): resolve dependency endpoints from env; don't block sta
410afda5 feat: Hyper provider discovery + registry + verifier types

Committed in helix_agent's own history but NOT yet reflected in main's gitlink — 2 commits (see Known gaps):

69a5ab8d fix(ollama): close goroutine-leak in CompleteStream on ctx cancel
a345c551 fix(tests/unit): wire the model source; stop reading recorder-def

submodules/tool_schema — 1 commit since the previously-pinned cb7bf675, fully pinned

8cec90b fix(i18n): embed translation bundles so lookups work from any dire

main — 6 substantive commits in 5725ee06..84853455 (revision 4 range)

04f88a23 items: re-derive the trackers from the DB and repair the segments
be5d56be gates: execute the three standing guards nothing was running (G30
69573772 items: file the seven unfiled findings from this session (HXC-249
b947276f items: file the two intake-layer findings reported in prose last
f10c9a1e review(r4): close the 8 findings from the round-4 review of be5d5
84853455 items(grpc-ports): file the two gRPC port defects found in helix_

(2816246e, the commit that wrote revision 3 of this document, also falls in this range and is omitted here as self-describing.)

submodules/helix_agent — full 66a3c1c6..HEAD range, 24 commits, now entirely pinned into main's gitlink (94e2dcc8)

This is the range revision 3 recorded only as “Known gap 14” (pin/checkout mismatch) and explicitly declined to enter. It is now fully described in the “Revision 4” section above.

```
16a8a1d1 fix(tests): stop reporting a wrong-service-on-port as a HelixAgent
d13132e3 fix(tests): derive the generator work dir instead of a foreign ab
6fbf6282 fix(redis): resolve one Redis endpoint everywhere – guard and sub
d6856cc9 test(timing): make 5 contention-sensitive assertions measure the
1796a588 test(identity): make the HelixAgent identity gate falsifiable, an
fe183ce4 test(redis): cover the AUTH+PING handshake the commit's own headl
34df0832 docs(redis): pin the three callers where guard and subject do NOT
159dd839 fix(mcp-servers): make the shipped Containerfile buildable again
816f37c2 fix(mcp): derive container MCP port and URL from one lookup, not
70e28ead fix(zen-cli): return the answer when the turn closes, and report
bc26122f fix(opencode-config): emit canonical model ids – the generator vi
32d2e8cb test(identity): pin the strict-mode env var's literal name, closi
526693cd docs(env): document HELIXAGENT_REQUIRED, and record why no make t
b88dadd9 fix(deps): tidy go.sum – 30 stale hashes and 2 structurally missi
1a55c8aa fix(validation): the complete-validation runner reported PASSED o
604a2f49 fix(cli-agent-configs): the generators aimed users at a different
886f72c0 test(validation): the readiness helper had no self-test – three m
ad3b5590 fix(validation): `grep -c ... || echo "0"` rendered "0 0 packages"
d2d70206 fix(grpc-server): our own gRPC server could not start while our o
ec95a277 fix(grpc-server): the fix moved the listener but left the banner
14a779fe docs(grpc): ten artifacts still published :50051, and the overrid
d17cee81 fix(cli-agent-configs): the endpoint was resolved for 2 generator
76b25796 fix(grpc-challenge): both branches recorded success, so it certif
94e2dcc8 fix(k8s): the manifests pinned a dead env var and a port nothing
```

Pre-existing documentation gap found, out of scope to close here. The

a345c551..66a3c1c6 range immediately preceding the 24 commits above — 32

further commits, including two dependency-advisory batches (dd8fcf4c,

846282b2 — the latter is the root cause b88dadd9's commit message cites for the go.sum drift this range fixed), several HXC-1xx/2xx security and boot-reliability fixes (HXC-221 API-key entropy, HXC-212 CORS allowlist, HXC-172 DNS-rebinding defence, HXC-228 slow-dependency boot wait, HXC-234 blind-guard closure), and a CRDT-based ensemble-sync schema addition — was **never itemized in any**

revision of this document (revisions 1–3 only ever referenced the endpoints a345c551 and 66a3c1c6, never what lies between them). This range is **already**

published on all four of the submodule's remotes (git merge-base --is-ancestor 66a3c1c6 refs/remotes/<r>/main succeeds for all four), so it is **not**

part of the “unpushed work” this task was scoped to document, and is reported here as a finding rather than backfilled — fully describing 32 more commits at this document’s established level of detail is a separate, larger documentation task. See “Known gaps”.

Known gaps / not yet verified

This section exists so this document does not read as more finished than the underlying work is. None of these were papered over.

1. **§11.4.185 manual QA-team confirmation has not been given.** No record of a manual QA sign-off for this scope exists anywhere in this repository. Per this project’s own governance, automated green is necessary but not sufficient — a manual QA pass is required before this scope can be called fully complete, and it has not happened.
2. **Desktop GUI packages do not build on this host.** Directly attempted go build ./applications/... in helix_code/ on this host and it fails with two real, host-environment errors, not code defects:
 - `github.com/go-gl/gl/v2.1/gl: pkg-config cannot find gl.pc.`
 - `github.com/go-gl/glfw/v3.3/glfw: fatal error: X11/Xlib.h: No such file or directory.` This is not a new discovery — a prior commit in this cycle (d6c05f76, already merged before this release’s commit range) documented this same gap and verified the README’s claimed error text is byte-identical to the real one on this host. Re-confirmed here, not re-litigated. Any package depending on the Fyne desktop GUI stack is affected.
3. **The zero-skip full integration test stack was not booted for the last sweep.** Checked directly (podman ps): the containers currently running on this host (helixcode-infra-postgres, -redis, -weaviate, -qdrant, -chromadb, -cognee, -ollama, etc., all systemd-managed via compose.helixcode-infra.yml) are the **production-like platform infra**, not the docker-compose.full-test.yml zero-skip test stack (which would run containers named helixcode-postgres-full, helixcode-redis-full, etc. — none of those are present). Any integration-test result gathered without first running make test-infra-up for that specific stack is environmental, not a confirmed pass or fail, until it is booted and the sweep re-run against it.
4. **The full go test suite has not concluded cleanly on a quiet host for this cycle.** docs/CONTINUATION.md’s own rev-18 audit (not independently re-run as part of this documentation-only task, to avoid adding more load to a host other agents are actively building/testing on) records one completed

helix_agent full-suite run on a contended host reporting 18 FAIL packages, and attributes this to ambient host contention (constitution §11.4.119) consistent with this project's own documented pattern — but that specific 18-package list has not itself been re-run on a quiet host and confirmed clean. Treat as **unconfirmed**, not cleared.

5. **helix_agent's goroutine-leak fix (69a5ab8d) and test fix (a345c551) are real, committed work but are NOT yet part of this release's pinned artifact.** Main's current gitlink for submodules/helix_agent points at 3268fd73; the submodule's own HEAD is two commits further, at 69a5ab8d. Until the gitlink is bumped and re-verified, this release does not actually ship the Ollama goroutine-leak fix even though the fix exists and is committed.
6. **helix_agent's working tree is dirty** (modified tracked files and untracked new test files observed at read time) and **2 commits ahead of its own configured remotes** — i.e. not yet pushed anywhere outside this host. This draft does not push anything.
7. **main is 8 commits ahead of every configured remote** (0 behind), i.e. also not yet pushed. This document commits only itself; per this task's constraints, nothing is pushed and no tag is created.
8. **Two commits carry the literal message Auto-commit** (56f7edf7, 1296d328) with no descriptive body. Real, substantive content was recovered from their diffstats (see the Infrastructure / Added tables above) rather than invented, but the commit messages themselves do not describe intent, verification, or evidence — a documentation-clarity gap at the commit-message layer, flagged rather than silently backfilled.
9. **No claim in this document was accepted from a stale project artifact without independent re-derivation of the specific git facts it depends on** (HEAD SHAs, gitlink pins, tag list, container names) — those were each re-checked directly against the live repositories at draft time. The one exception is item 4 above, which is explicitly attributed to docs/CONTINUATION.md rather than re-run, and is labeled unconfirmed for exactly that reason.
10. **helix_agent has moved further since this draft's stated pin (69a5ab8d), adding a third sibling to the same defect class this document already tracks.** Commit e2f42b12 (fix(llm,database): prioritize cancellation in racy select statements) fixes exactly the two “sibling” locations dd3c0c3b's commit message names as instances of the same unprioritized-select pattern — internal/llm/lazy_provider.go's createProviderWithContext and internal/database/debate_log_repository.go's StartCleanupWorker — with the same RED/GREEN polarity-evidence discipline (§11.4.115). It is real, committed work, but as of this revision it is **not yet part of this release's pinned**

artifact: main's gitlink for submodules/helix_agent still points at 3268fd73, four commits behind helix_agent's own current HEAD (3268fd73 → a345c551 → 69a5ab8d → e2f42b12). This compounds Known gap 5 rather than replacing it — do not treat 69a5ab8d as the current unpinned tip; re-derive submodules/helix_agent's actual HEAD before citing it, per this document's own opening warning.

Known gaps added or re-measured in revision 3

Gaps 1–10 above were written against the revision 1–2 range. Re-checked on 2026-08-09; the following supersede or extend them.

1. **Three standing guards written this cycle are registered in no runner.** Measured: `scripts/verify-all-constitution-rules.sh` invokes **21** `scripts/gates/` entries and **0** `scripts/testing/guard_hxc*.sh`. There is no auto-discovery glob. So `guard_hxc229_gateway_release_mode.sh`, `guard_hxc233_completion_path_live.sh` and `guard_hxc244_health_components_registered.sh` all exist, are individually falsifiable, and **never execute**. §11.4.135 asks for a standing suite; these are standing files. This is the direct reason HXC-229 was left open rather than closed in 7bd80ff6.
2. **db-to-md silently drops two items.** `workable-items` validate reports HXC-247 and HXC-248 have no `doc_segments` row, and a regeneration to a scratch path confirms it empirically: both appear **0** times in the regenerated `Issues.md`, while every other item appears. 2 of 442 items. Pre-existing — reproduced identically against a pre-change backup — and deliberately **not** silently repaired, because synthesising the missing segments is adjacent to HXC-245's failure mode and deserves its own item.
3. **13 open items carry no severity, so §11.4.132 risk-ordering cannot rank them.** 196 rows have an empty severity, but 183 of those are already closed, where severity ranks nothing. The actionable set is the 13 open ones: HXC-227, 229, 231, 232, 233, 234, 235, 236, 239, 240, 241, 242, 243. No rule exists in this repository to derive a severity — the schema itself says *"informational only; not closed-set"*, with no CHECK and no default — so these were left alone rather than invented (§11.4.6).
4. **submodules/helix_agent's recorded pin and its working checkout disagree.** `git submodule status` shows a + prefix: the pin is 66a3c1c6, the checkout is d6856cc9. Three other agents were actively editing that submodule while revision 3 was written, so it was not entered and the delta is **not characterised here**. HXC-234 could not be resolved past ambiguous for the same reason — its fix lives inside that submodule.

5. **The release position is 3 ahead / 0 behind on all four remotes**, measured *after* a real `git fetch --all --prune`. Supersedes gap 7's "8 ahead". The three local commits are revision 3's own tracker work (1f177495, 7bd80ff6, 5725ee06). Nothing was pushed and no tag was cut.
6. **Two further Auto-commit commits** landed in the revision 3 range (c209bff2, 29696107), extending gap 8. Additionally the pinned `helix_llm` head 3d789cb itself carries the literal message Auto-commit — it was verified by content (it contains 8260cf8), not by its subject.
7. **§11.4.185 manual QA-team confirmation is still not given** — unchanged from gap 1, and now load-bearing for §11.4.238, which this range cascaded: manual QA is a confirmation step that must find nothing new, and it has not been run at all for this scope.

Known gaps added or re-measured in revision 4

Gaps 1–17 above were written against revisions 1–3. Re-checked on 2026-08-10/11; the following supersede, resolve, or extend them.

1. **Gap 14 is RESOLVED.** `submodules/helix_agent`'s recorded pin and its working checkout no longer disagree — main's gitlink is 94e2dcc8, which is exactly the submodule's own current HEAD as of 84853455. The `+ prefix git submodule status` previously showed is gone.
2. **The CLI-agent config port fan-out (604a2f49, d17cee81) is landed in source for every currently-wired call site, but is NOT confirmed complete against every call site that exists.** d17cee81's own commit message already flags three exported-but-currently-uncalled functions in the `llms_verifier` submodule (`DefaultGeneratorConfig()`, `DefaultMCPServers()`, `ContainerizedMCPServers(host)`, the last ignoring its own `host` argument for six `HelixAgent` URLs) as a residual hardcoded source. **This documentation pass independently verified a concrete, in-tree instance of exactly that risk:** `submodules/helix_agent's own internal/services/cli_agent_config_exporter.go` (`NewCLIAgentConfigExporter`) constructs its generator directly from `cliagents.DefaultGeneratorConfig()` — the same hardcoded-:8100 default every fixed call site in this range now avoids — with no port resolution at all. As of this documentation pass, `CLIAgentConfigExporter` has **no production caller anywhere in the tree** (`grep -rl "CLIAgentConfigExporter" --include="*.go" .` returns only its own source file and its own test file), so it does not affect the shipped binary *today*. It is nonetheless a landed liability: the moment any future commit wires that exporter into a running path (its name and doc comment both

suggest it was written to be wired in), it will silently reproduce the exact wrong-port defect this entire range of work exists to fix, and no test in this range would catch it, because no test in this range exercises that file. **Do not describe the port fan-out as “fixed” without this caveat — it is fixed for the paths proven reachable and enumerated in this range’s evidence (48/48 CLI-agent generators via the per-agent enumeration in d17cee81), not proven complete against every exported code path that could construct a generator.**

3. **The gRPC port fan-out (d2d70206, ec95a277, 14a779fe, 76b25796, 94e2dcc8) is landed in source and proven against unit/guard tests, but has NOT been verified against a rebuilt, redeployed HelixAgent artifact on a live host in this documentation pass.** Each individual commit cites its own RED/GREEN evidence captured against that commit’s own build, which is real evidence of that specific change — but no commit in this range re-runs the full chain end-to-end (build → deploy → dial the published address → get a real gRPC response) after all five land together. This document was not the place to perform that verification (out of scope: documentation-only, no source/script changes, and submodules/helix_agent/challenges/* and cmd/helixagent/* were explicitly off-limits per this task’s own hard constraints because other agents are actively working there). Treat the gRPC fan-out the same way as gap 19: landed in source, verification on a clean rebuilt/redeployed target pending, per §11.4.108’s SOURCE → ARTIFACT → RUNTIME → USER-VISIBLE layering.
4. **A 32-commit pre-existing documentation gap was found and is reported, not backfilled.** submodules/helix_agent’s a345c551..66a3c1c6 range — immediately preceding this revision’s 24-commit range, and already published on all four of the submodule’s remotes — was never itemized in any revision of this document. It contains real, substantive fixes (two dependency-advisory batches, several HXC security/reliability items, a CRDT ensemble-sync schema addition). It predates the “unpushed work” scope this task was assigned, so it is out of scope to fully document here; see the “Per-repo commit inventory” section above for the measured range and citation. Recommended as a follow-up documentation task, not attempted in this pass.
5. **submodules/helix_agent’s working tree carries one modified-but-uncommitted file** (releases/.version-data/helixagent.last-hash) at the time this document was written, and other in-flight, uncommitted work was observed at the start of this task in challenges/scripts/, k8s/base/*, and internal/ports/published_grpc_port_test.go before other agents’ concurrent commits (76b25796, 94e2dcc8) landed some of it. Per this task’s explicit hard constraints, none of submodules/helix_agent/cmd/helixagent/*, k8s/*, or challenges/* was touched**

by this documentation pass, and this document describes only what those other streams had already committed** by the time each fact below was measured — not their still-uncommitted state, which by definition is not yet part of any release artifact.

6. **Ten tracker items opened or updated in this range remain Queued** (HXC-247, 248, 250, 251, 254, 255, 256, 257, 258, 259) plus **HXC-260 and HXC-261**, filed by the final commit in this range (84853455) and also Queued. None was closed by this documentation pass — closing a tracker item requires its own `workable-items close` invocation with captured evidence, which is a code/tooling action outside this task’s documentation-only scope.
7. **§11.4.185 manual QA-team confirmation is still not given** — unchanged from gap 17, and this range adds more scope (the gRPC and CLI-config fan-outs) that would need it before either could be called fully complete.

Independent review

Per constitution §11.4.142 (every change gets an independent review, no exception) and §11.4.194 (exhaustive all-scenario review), this document received a full independent review after its original commit (8bdac5b0) landed. The authoring agent had been killed by an API session limit immediately after committing and never reported, so this changelog had zero review until now.

Method: every cited short SHA in the Fixed / Added / Changed / Infrastructure tables and the per-repo commit inventories was checked to exist and to actually do what its entry claims, via `git show --stat` and full `git show` against this repository, submodules/helix_agent, and submodules/tool_schema — not accepted on the strength of the subject line alone. Quantitative claims (assertion counts, PASS/FAIL tallies, live scores, frame counts) were spot-checked against the cited commits’ own bodies. `git log --oneline 8bdac5b0..HEAD` was run in this repo (empty — main has not moved) and, using the changelog’s own stated submodule pins as the base, in submodules/helix_agent (one further commit, e2f42b12 — see Known gap 10) and submodules/tool_schema (none). The `.env` HELIX_RELEASE_PREFIX resolution, the §11.4.44 revision header, and the “3 desktop packages fail to build” claim (applications/desktop, applications/aurora_os, applications/harmony_os — confirmed all three import the Fyne stack, so all three are genuinely covered by `go build ./applications/...` and by the stated X11/OpenGL failure) were all independently re-verified rather than taken on trust.

Finding (blocking, now fixed in revision 2): commit dd3c0c3b — this changelog’s own direct parent commit, committed roughly 90 seconds before 8bdac5b0 — was completely absent from every section of the original draft

despite being real, substantive, in-scope work covered by the document's own stated data-collection window. It has been added to the Fixed table, the per-repo commit inventory, and cross-referenced against its `helix_agent` sibling fix (Known gap 10).

Verdict: no invented history was found anywhere in this document — every other cited SHA, quantitative claim, and Known-gaps disclosure checked out exactly as stated. With the one finding above corrected, this document is independently confirmed accurate as of that revision (2026-07-28T15:21:59Z).

Revision 3 (2026-08-09)

Why it was needed: the document was pinned to a HEAD 11 days and 231 commits stale, and covered none of the work in that range.

Method: the range was re-derived from `git log / git show`, never from memory or from any prior project artifact. Every short SHA newly cited above was checked to exist and to carry the subject attributed to it — **51 main-repo SHAs verified present via `git cat-file -e`, 0 missing**, plus `helix_llm 3d789cb` and `8260cf8` and `constitution 177f2b0` in their own repositories. The two `helix_agent` values (`66a3c1c6`, `d6856cc9`) are quoted from main-repo data only — `git ls-tree HEAD` and `git submodule status` — precisely because that submodule was not entered. Counts are pasted command output, not estimates: 231 commits in `c29e1dcc` . . HEAD, 2 of them the literal Auto-commit, 0 merges, 249 since the tag. Remote positions were measured **after** an actual `git fetch --all --prune` rather than from tracking refs, because this range contains a commit (`f0dd7069`) whose own message documents a tracking ref reporting `behind=0` before a fetch and `behind=3` after.

Finding 1 (corrected here): revision 2 introduced a counting error while fixing a real omission. It raised “commits since the last tag” from 18 to 19 to account for `dd3c0c3b`. `git rev-list --count 5ab97d8c . . c29e1dcc` returns 18, and `dd3c0c3b` is provably *after* `c29e1dcc` — it belongs to the revision 3 range. The commit did need adding to the Fixed table; the count did not need changing. Both are now correct and reconcile: $18 + 231 = 249$.

Finding 2: a claim in the revision 1–2 pin table could not have been true as written and is superseded. It described `submodules/helix_agent` as pinned at `69a5ab8d`. The pin recorded in HEAD's tree today is `66a3c1c6`, and the working checkout is a third value, `d6856cc9`. Rather than reconcile a stale figure, revision 3 states all three positions and marks the delta uncharacterised (Known gap 14), because the submodule was deliberately not entered.

Verified rather than assumed: that the pinned `helix_llm (3d789cb)` actually contains `8260cf8`, the HXC-244 health fix — `git merge-base --is-ancestor` confirms it, so a fresh clone gets the fix. This mattered because this same range

contains two separate incidents (f0dd7069, 204bc7b4) of pins that did not carry the fixes attributed to them.

Honest boundary (§11.4.6): revision 3 has itself had **no** independent review — it is authored by the same agent that produced the tracker commits it describes. Its factual claims are individually re-derivable from the commands cited, but the §11.4.142 independent-reviewer seam has not been crossed for this revision. Treat the revision 3 sections as verified-by-construction, not peer-reviewed.

Revision 4 (2026-08-10/11)

Method. This revision was authored as a dedicated documentation-and-version-metadata task, structurally separate from the agents that produced the commits it describes (a different task, not merely a different commit, but not the formal §11.4.142/§11.4.194 code-review dispatch either — see the honest boundary below). Every SHA cited in the revision 4 tables was read from a real `git show/ git log -1 --format=%B` invocation against `submodules/helix_agent` or this repository — none was copied from a prior summary or session note. Every tracker status cited was read from `docs/workable_items.db` via `sqlite3 -readonly` at documentation time, not inferred from any commit message's own claim about what it fixed (which is precisely how revision 3's "Fixed" table headers for HXC-229/233/235 were found, in this revision, to be describing source-level fixes without the tracker items actually being closed — see the "Tracker / workable-items" table above).

Finding 1: one call site independently verified beyond what d17cee81's own commit message disclosed. d17cee81 named the residual hardcoded risk as living entirely in the `llms_verifier` submodule (out of this task's repository scope). This revision additionally found, by reading `submodules/helix_agent/internal/services/cli_agent_config_exporter.go` directly, that the same unresolved default is also reachable from a function *inside* `submodules/helix_agent` itself (`NewCLIAgentConfigExporter` → `cliagents.DefaultGeneratorConfig()`), currently unwired to any production entry point. This is reported as Known gap 19 with the caveat that "unwired" was itself verified (a repo-wide `grep` for the type name), not assumed.

Finding 2: three more unpushed commits landed on live hosts during this task's own execution window, confirming the continuous-parallel-work pattern this document has flagged since revision 1. The task that requested this document was briefed against 9 main-repo / 22 submodule commits; by the time the git facts below were gathered, the true count was 10 / 24 (one additional main commit, 84853455; two additional submodule commits, 76b25796 and 94e2dcc8 — precisely the two fix commits 84853455 itself references). All three are documented in full above rather than silently omitted or silently included without acknowledging the discrepancy.

Honest boundary (§11.4.6). Revision 4, like revision 3, has **not** crossed a formal §11.4.142/§11.4.194 independent-code-review seam — this task was scoped to documentation and version metadata only, explicitly excluding source, test, and script changes, so no code review of the underlying diffs (beyond the spot-checks and the one live-file verification described above) was performed or claimed. Treat the revision 4 sections as verified-by-construction against real git and database state, not as a substitute for the §11.4.125/§11.4.142/§11.4.194 review the underlying code changes still owe before they can be described as fully done.

Revision 5 (2026-08-10/11)

Why it was needed. The coordinator reviewed revision 4 and identified that its treatment of HXC-229/233/235 risked a §11.4 documentation-layer overclaim in the opposite direction from what it was trying to avoid: having correctly declined to describe these three as “Fixed” (matching revision 3’s table-header ambiguity), revision 4’s tracker table instead pointed toward describing them the same way as the CLI-agent/gRPC port fan-outs (“landed in source, verification pending”) — which would have **understated** real, already-captured, git-tracked runtime evidence that this revision found by reading `docs/workable_items.db’s item_history` and `test_diary` tables directly, and the `docs/qa/` evidence paths they cite.

Method. Per-item: confirmed the source SHA; located and read the standing-guard script or test file directly (not inferred from a commit message, per the coordinator’s explicit §11.4.226 instruction); ran the HXC-229 and HXC-233 guards live, read-only, against the real running host (`bash scripts/testing/guard_hxc229_gateway_release_mode.sh` and `.../guard_hxc233_completion_path_live.sh`); queried `item_history` and `test_diary` via `sqlite3 -readonly` for every event and evidence path recorded against these three IDs; located the independent-review commit (c8171122) these diary rows trace back to and read its full body for the per-item reasoning; located the two later evidence commits (be5d56be adding the HXC-229/233 guards, and the pre-existing f2e12570/FINDING.md redeploy for HXC-235) that each address the specific blocking reason the independent review recorded; and confirmed via `git ls-files` that every cited `docs/qa/` path is actually tracked, not merely present on disk.

Finding, restated for the record. All three items have genuine, tracked runtime evidence — this is not the “verification never captured, tracker correctly Queued” case the investigation was framed to expect as most likely, nor is it a simple “tracker is stale, should say Fixed” case either. For HXC-229 and HXC-235, the tracker’s own independent-review process recorded one specific, evidenced reason each stayed open, and that specific reason has since been resolved by later work — without a follow-up review or diary entry recording the resolution. For HXC-233, no closure review was ever attempted at all. Full per-item detail, with

evidence paths and exact timestamps, is in “Revision 5 — HXC-229/233/235 tracker-vs-runtime-state reconciliation” above.

Honest boundary (§11.4.6). This revision changed no item’s status in `docs/workable_items.db` — per the coordinator’s explicit instruction, that routing decision belongs to whoever owns the sanctioned `workable-items` close step, not to this documentation task. The live guard re-runs for HXC-229 and HXC-233 were performed by this task (read-only, no state mutation); HXC-235 has no standalone guard to re-run, so its finding rests on the two already-committed, tracked evidence artifacts rather than a fresh observation. Like revisions 3 and 4, revision 5 has not itself crossed a formal §11.4.142 independent-code-review seam.

A structural repair, unrelated to the reconciliation itself. While locating section anchors for this revision, a prior editing defect in revision 4 was found and fixed: the `## Per-repo commit inventory` heading and its first subsection heading (`### main (this repo) – 18 commits...`) had been accidentally dropped when revision 4’s content was spliced into the document, leaving the revisions 1–2 `commit-list` content orphaned under no heading at all. Restored verbatim; no content was lost, only its heading, and `git diff` on this commit shows the two-line addition that fixes it.

Sources verified

All SHAs, commit subjects, commit bodies, tag names/dates, gitlink pins, container names, and the `go build` error text above were read directly from this repository, submodules/`helix_agent`, and submodules/`tool_schema` via `git log`, `git show`, `git tag`, `git ls-tree`, `podman ps`, and a direct `go build` invocation on 2026-07-28. No claim in the Fixed / Added / Changed tables above is unsourced from a real commit; anything that could not be traced to a commit was left out rather than inferred.

Revision 3 (2026-08-09). The `c29e1dcc..5725ee06` range was derived with `git rev-list --count`, `git log --oneline/--format`, `git show --stat`, `git log -1 --format=%B` on every commit cited, `git merge-base --is-ancestor` for the ancestry claims, `git ls-tree HEAD <path>` and `git submodule` status for the pins, `git fetch --all --prune` followed by `git rev-list --count refs/remotes/<r>/main..HEAD` for the remote positions, `git ls-files` to confirm cited evidence directories are actually **tracked** rather than merely present on disk, and `sqlite3 -readonly plus workable-items validate /sync db-to-md` for every tracker figure. The constitution anchors were read from `constitution/Constitution.md` at the pinned commit and confirmed to be genuine `###` block-openers, not incidental mentions. File sizes were taken with `wc -c`, not `stat`, which misreports on this host. Nothing in the revision 3 sections is carried over from a prior document without re-derivation.

Revision 4 (2026-08-10/11). The 5725ee06..84853455 main-repo range and the full 66a3c1c6..HEAD (24-commit) submodules/helix_agent range were derived with `git log --format='=====%n%H%n%ci%n%B' <range> --reverse` (captured to a scratch file and read in full, not sampled), `git show --stat --format=fuller` and `git log -1 --format=%B` on every individually-cited SHA, `git rev-list --count` for every range-size claim (including the a345c551..66a3c1c6 32-commit gap and the 256-commits-since-tag figure), `git merge-base --is-ancestor` for the “already published on all four remotes” claim about that gap, `git ls-tree HEAD constitution submodules/helix_agent submodules/tool_schema submodules/helix_llm` and `git submodule status` for every pin (confirming the pin/checkout match that resolves Known gap 14), `git fetch --all --prune` on both repositories followed by per-remote `git rev-list --count` for every ahead/behind figure, `git tag -l` on both repositories for the §11.4.151 compliance section (including the two main-repo legacy tags this task’s briefing did not mention), `sqlite3 -readonly docs/workable_items.db` for every tracker status/count cited (including the 453→455 item-count reconciliation), and a direct `grep -r1 "CLIAgentConfigExporter" --include="*.go" .` plus a direct read of `submodules/helix_agent/internal/services/cli_agent_config_exporter.go` for the Known-gap-19 finding. Nothing in the revision 4 sections is carried over from the task’s own briefing without independent re-derivation — the briefing’s stated commit counts (9 main / 22 submodule) were themselves found stale by 1 and 2 commits respectively during this process and the document reflects the re-measured counts (10 / 24), not the briefed ones.

Revision 5 (2026-08-10/11). The HXC-229/233/235 reconciliation was derived with: `sqlite3 -readonly docs/workable_items.db` against `items` (status, severity), `item_history` (event log — confirmed all three show only an Opened event, no Fixed/Implemented/Completed event, no `evidence_path`), and `test_diary` (`entry_id`, `atm_id`, `date_time`, `tested_by`, `result`, `result_detail`, `evidence_path`, queried directly against `entry_id` IN (34,35,36) and separately confirming zero rows exist for `atm_id='HXC-233'`); `git log --all --oneline -i --grep="HXC-229" / "HXC-233" / "HXC-234"` to locate the independent-review commit (c8171122) and confirm no equivalent exists for HXC-233; `git log -1 --format=%B c8171122` read in full for the per-item reasoning; direct execution, read-only, of `scripts/testing/guard_hxc229_gateway_release_mode.sh` and `scripts/testing/guard_hxc233_completion_path_live.sh` against the live host (both exit 0, output quoted verbatim above); `git ls-files` against every `docs/qa/` directory cited (hxc229_gin_debug_mode_20260807T000000Z, hxc233_routing_fix_20260808T140500Z, hxc235_deploy_20260808T135200Z, independent_review_hxc229_234_235_20260808T132900Z) to confirm each is

tracked, not merely present on disk; and a direct read of docs/qa/hxc233_routing_fix_20260808T140500Z/FINDING.md and docs/qa/hxc235_deploy_20260808T135200Z/FINDING.md in full. The submodules/helix_llm test-file existence check (hxc235_embedding_semantic_signal_test.go, hxc235_semantic_signal_test.go) was confirmed via that submodule's own git ls-files, read-only — no file in that submodule, or in submodules/helix_agent (explicitly off-limits per this task's scope), was modified. No item's status in docs/workable_items.db was changed by this revision.